Prepare training, validation and test data lists

Randomly select 10% of the dataset as validation and 10% as test.

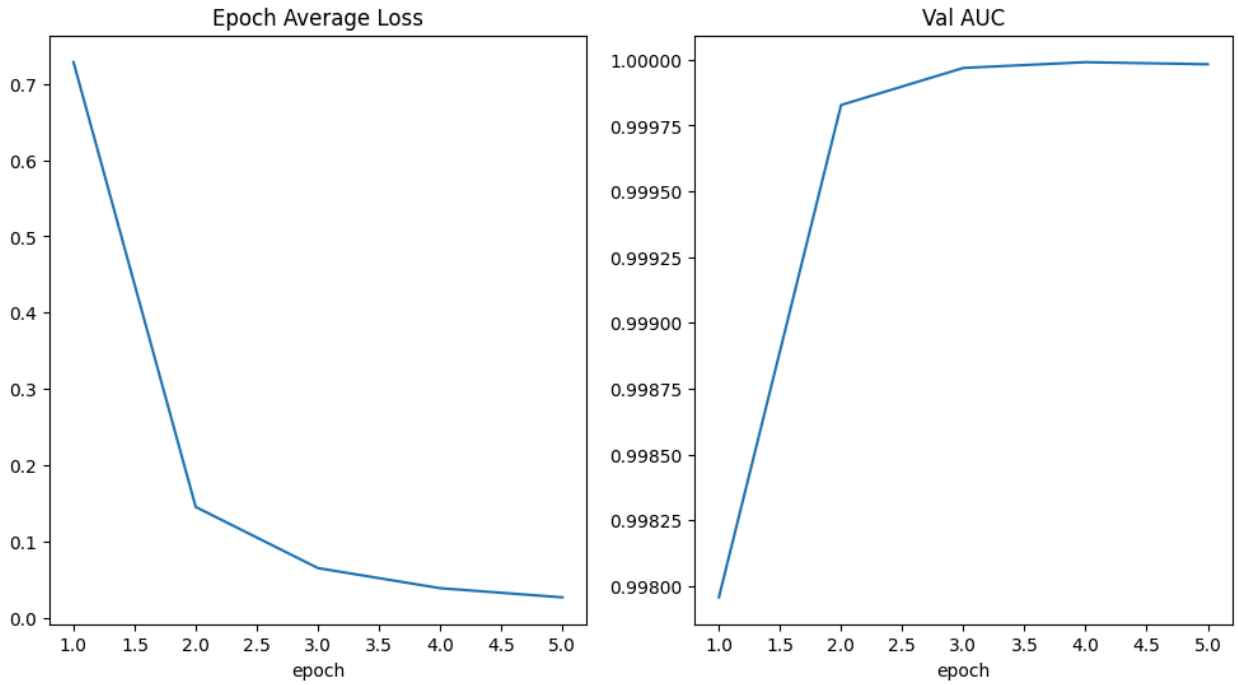
```
val_frac = 0.1
test_frac = 0.1
length = len(image_files_list)
indices = np.arange(length)
np.random.shuffle(indices)

test_split = int(test_frac * length)
val_split = int(val_frac * length) + test_split
test_indices = indices[:test_split]
val_indices = indices[test_split:val_split]
train_indices = indices[val_split:]

train_x = [image_files_list[i] for i in train_indices]
train_y = [image_class[i] for i in train_indices]
val_x = [image_files_list[i] for i in val_indices]
val_y = [image_class[i] for i in val_indices]
test_x = [image_files_list[i] for i in test_indices]
test_y = [image_class[i] for i in test_indices]

print(f"Training count: {len(train_x)}, Validation count: " f"{len(val_x)}, Test count: {len(test_x)}")
```

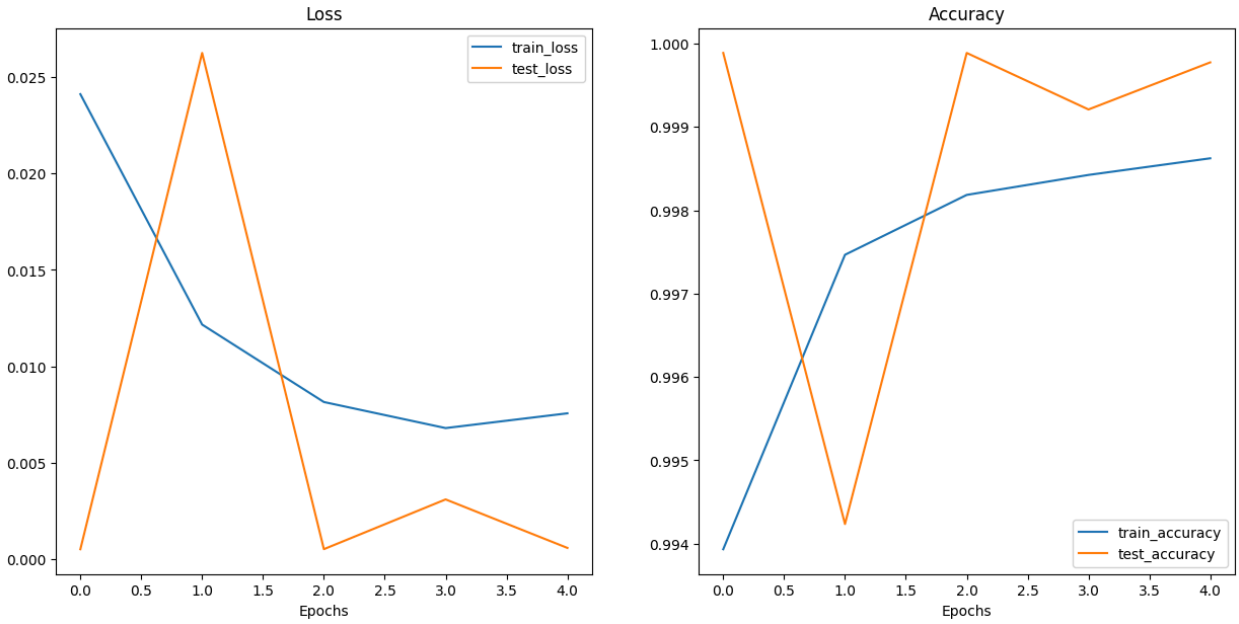
```
Training count: 47164, Validation count: 5895, Test count: 5895
```

```
# Create directories for train, validation, and test datasets
train_dir = os.path.join(root_dir, "train")
test_dir = os.path.join(root_dir, "test")

# Remove the folders if they already exist
if os.path.exists(train_dir):
    shutil.rmtree(train_dir)
if os.path.exists(test_dir):
    shutil.rmtree(test_dir)

for folder in [train_dir, test_dir]:
    for class_name in class_names:
        os.makedirs(os.path.join(folder, class_name), exist_ok=True)

# Function to copy images to respective directories
def copy_images(indices, target_dir):
    for idx in indices:
        src = image_files_list[idx]
        class_name = class_names[image_class[idx]]
        dst = os.path.join(target_dir, class_name, os.path.basename(src))
        shutil.copy(src, dst)

# Copy images to train, validation, and test directories
copy_images(train_indices, train_dir)
copy_images(val_indices, test_dir)

print(f"Images organized into folders: {train_dir}, {test_dir}")
```

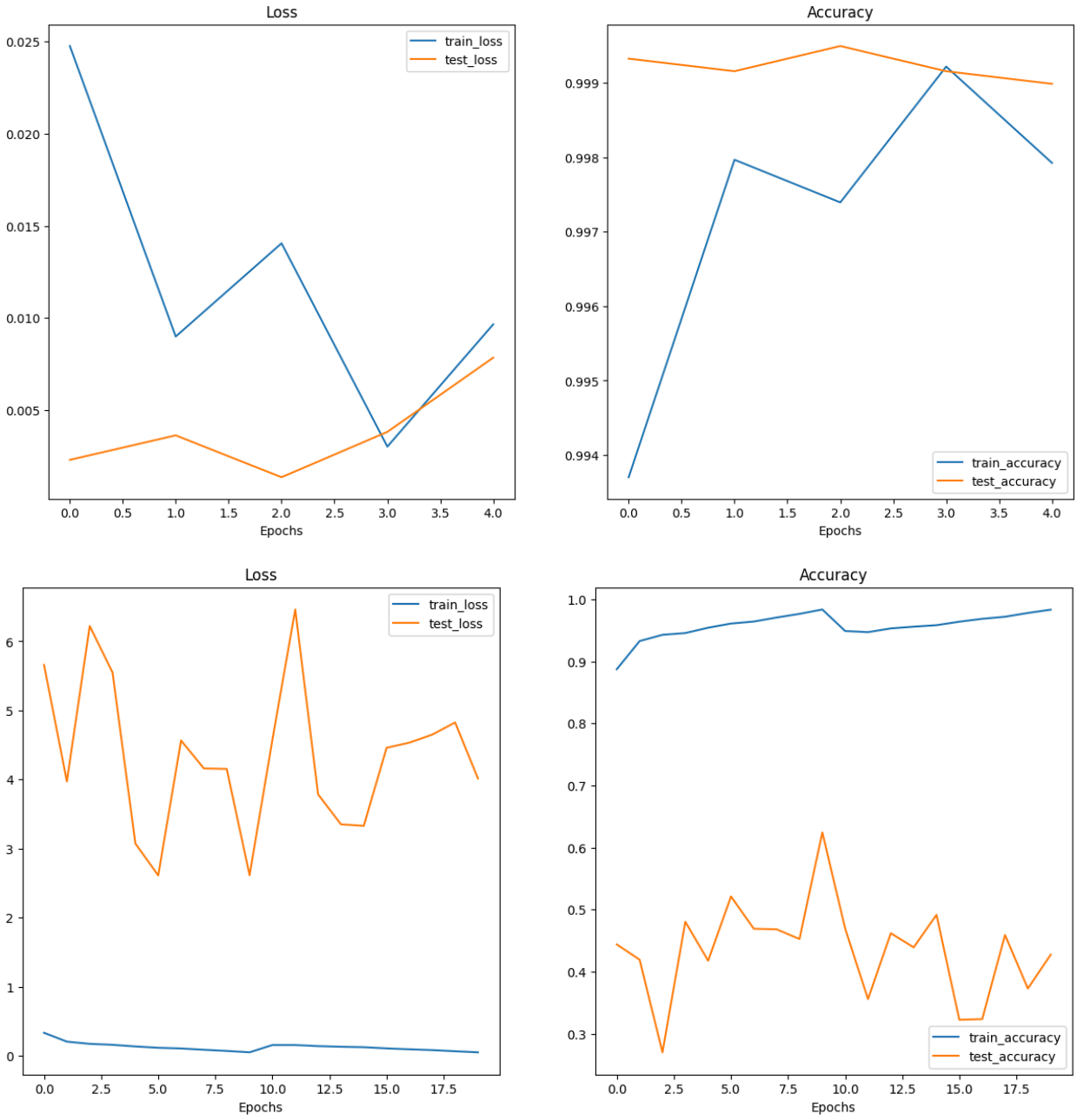
```
Training count: 50111, Test count: 8843
Images organized into folders: data/train, data/test
```

First, during the re-training process using the TIMM library, I modified the dataset partitioning logic. Instead of relying on the original MonoAI-style random split, I adopted TIMM's indexed slicing approach using the intervals idx[:test_split], idx[test_split:val_split], and idx[val_split:]. This allowed for a more deterministic and reproducible dataset division. As TIMM requires datasets to be structured in directory-based format, I reorganized the data accordingly. The code created separate TRAIN and TEST directories, and each image was moved to its respective folder based on its class label, maintaining the class-wise structure during the transfer. This restructuring enabled TIMM to load the images efficiently during training and evaluation.

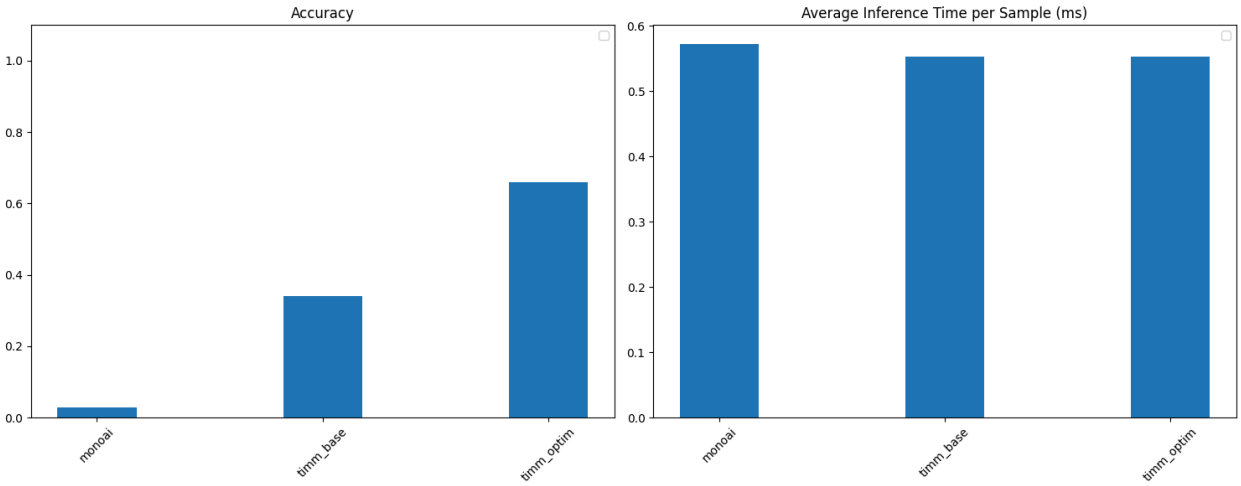




After training the baseline TIMM model provided in the sample notebook CMPE_pytorch8_2024Fall_timm.ipynb, I applied a series of optimization and tuning techniques. The training process led to a noticeable reduction in the loss value over epochs, as shown in the plotted training curves. However, despite the improved loss, the accuracy metrics showed limited improvement. This suggests that while the model became better at minimizing the training objective, its ability to generalize to unseen data remained constrained, possibly due to overfitting or suboptimal hyperparameters.



Following the optimization strategies demonstrated in the provided example code, I incorporated a learning rate scheduler into the TIMM training pipeline and replaced the default optimizer with AdamP, which is known for better generalization in vision tasks. These changes significantly increased the overall training time due to additional computation and convergence steps. Nevertheless, the modified training regime yielded positive results — both the loss and accuracy metrics showed consistent improvements compared to the previous setup. The learning rate scheduler helped stabilize training, and AdamP likely contributed to better generalization.



Out[15]:

	Model	Accuracy	Inference Time (ms)
0	monoai	0.0278	0.57
1	timm_base	0.3411	0.55
2	timm_optim	0.6601	0.55

Next, I exported the trained models from the three different training methods into the standardized **ONNX (Open Neural Network Exchange)** format to facilitate deployment and cross-platform testing. A subset of images was randomly selected from the original dataset to evaluate the predictive performance of all three models. The test results clearly indicate a substantial improvement in classification accuracy in the later models compared to the initial baseline model. Additionally, the inference latency remained stable across all models, ensuring that the increased accuracy did not come at the cost of runtime efficiency or responsiveness.